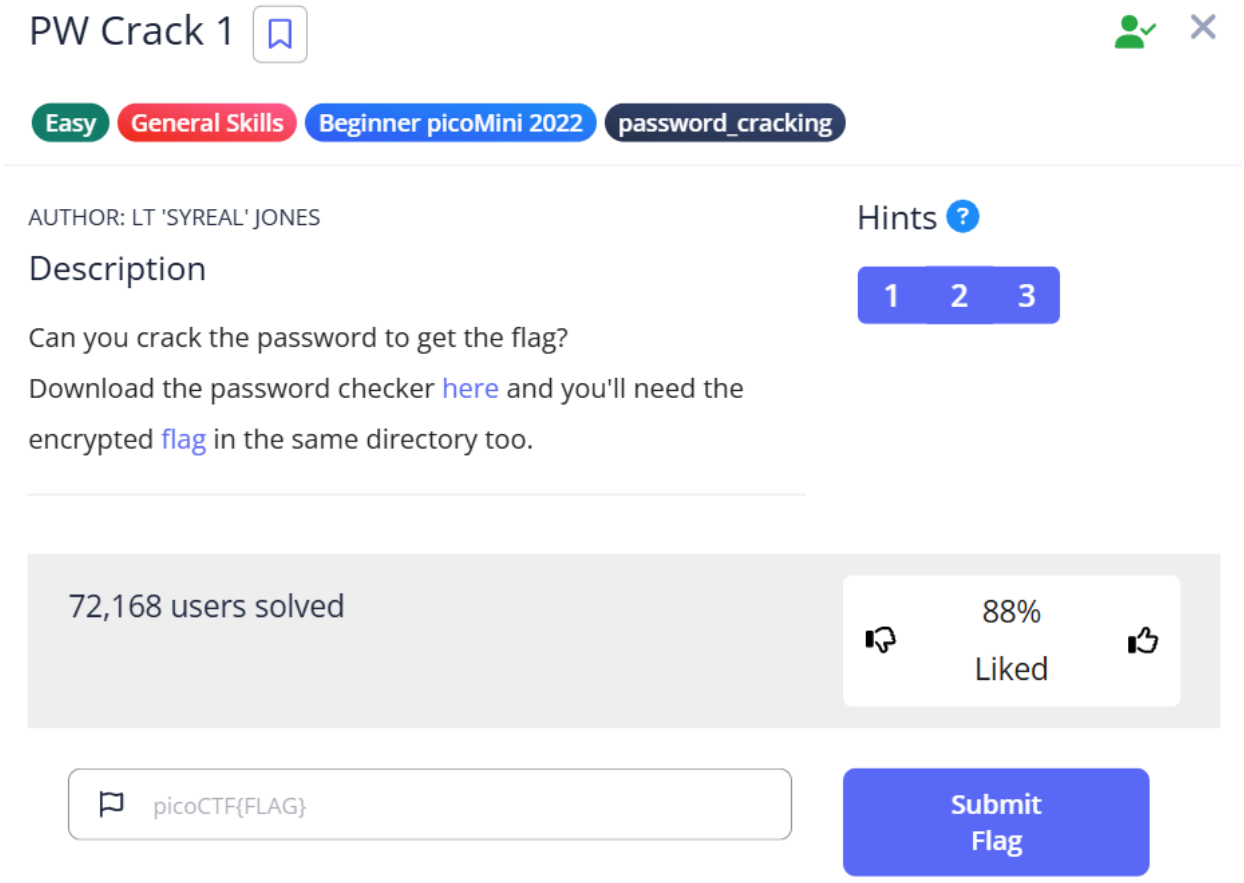
Saint Columban College
College of Computing Studies
Pagadian City

PW Crack 1



Author: The Analyst: Hyposelenia

Challenge: PW Crack 1

Category: General Skills

Date: 01/27/26

I. Objective

The objective of this challenge is to analyze a provided Python script to determine the correct password required to decrypt an encrypted file and retrieve the hidden flag.

II. Background

Password-related challenges in Capture The Flag (CTF) competitions introduce participants to **basic password analysis, code inspection, and file decryption concepts**. Rather than performing complex brute-force attacks, beginner challenges often require **reading and understanding source code** to identify how a password is verified.

In this challenge, a Python script is used to check a user-supplied password. If the correct password is entered, the script decrypts the contents of an encrypted text file and displays the flag. Understanding how scripts reference files within the same directory and how conditional statements work is essential to solving this challenge.

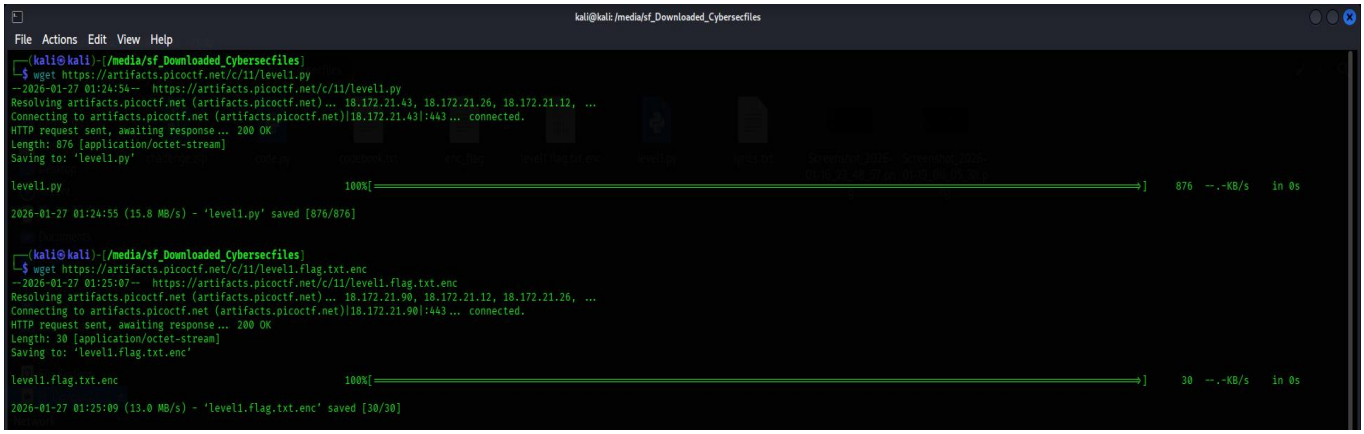
III. Tool Used

- **Oracle VirtualBox (Kali Linux)** – Used as the Linux environment
- **Linux Terminal** – Used to execute decoding commands
- **Python 3** – Used to execute the provided Python script
- **nano** – Used to view and inspect the Python source code

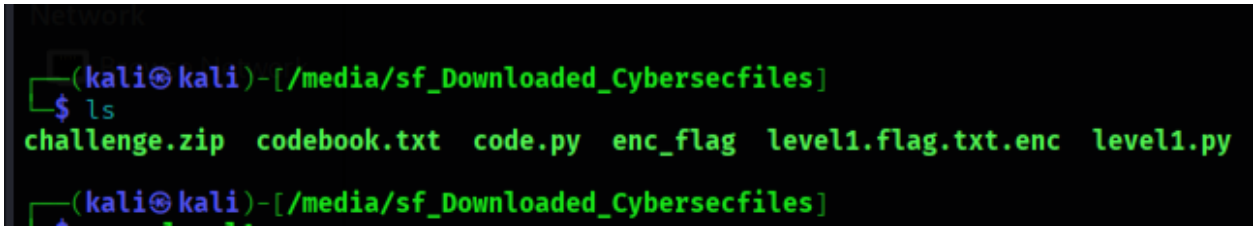
IV. Methodology

1. Opened Oracle VirtualBox and launched Kali Linux.

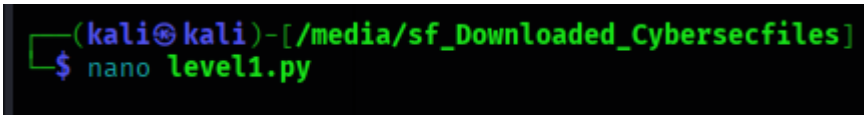
2. Downloaded the provided challenge files, which included a Python script and an encrypted text file. (This time I used its link address to DL)



3. Ensured that both files were placed in the **same directory**, as required by the script.
4. Opened the Linux Terminal and navigated to the directory containing the files.



5. Used the **nano** command to view the Python script and analyze its contents.



6. Observed an if condition within the script that checks whether the entered password matches a specific value. (CTRL + X to exit)

```
 GNU nano 8.4                                                                level1.py
### THIS FUNCTION WILL NOT HELP YOU FIND THE FLAG --LT ########################
def str_xor(secret, key):
    #extend key to secret length
    new_key = key
    i = 0
    while len(new_key) < len(secret):
        new_key = new_key + key[i]
        i = (i + 1) % len(key)
    return "".join([chr(ord(secret_c) ^ ord(new_key_c)) for (secret_c,new_key_c) in zip(secret,new_key)])
###############################################################################


flag_enc = open('level1.flag.txt.enc', 'rb').read()


def level_1_pw_check():
    user_pw = input("Please enter correct password for flag: ")
    if( user_pw == "1e1a"):
        print("Welcome back... your flag, user:")
        decryption = str_xor(flag_enc.decode(), user_pw)
        print(decryption)
        return
    print("That password is incorrect")


level_1_pw_check()
```

7. Identified that when the correct password is entered, the script prints a welcome message and decrypts the contents of the encrypted text file.
8. Executed the Python script using the python3 command.

```
┌──(kali㉿kali)-[/media/sf_Downloaded_Cybersecfiles]
└─$ python3 level1.py
```

9. Entered the correct password when prompted.

```
┌──(kali㉿kali)-[/media/sf_Downloaded_Cybersecfiles]
└─$ python3 level1.py
Please enter correct password for flag: 1e1a
```

10. Retrieved and submitted the flag displayed by the script.

V. Result

picoCTF{545h_r1ng1ng_fa343060}

```
┌──(kali㉿kali)-[/media/sf_Downloaded_Cybersecfiles]
└─$ python3 level1.py
Please enter correct password for flag: 1e1a
Welcome back... your flag, user:
picoCTF{545h_r1ng1ng_fa343060}
```

VI. Explanation

This challenge demonstrates the importance of **code review over brute-force methods**. Instead of attempting to guess or crack the password blindly, inspecting the Python script revealed the exact value required to pass the password check.

Placing both the Python script and the encrypted text file in the same directory was essential, as the script reads the encrypted file using a relative file path. If the files were stored in different directories, the script would fail to locate the encrypted file.

From a cybersecurity perspective, this challenge highlights why **hard-coding passwords inside source code is insecure**. Anyone with access to the code can easily discover the password, bypass authentication, and access protected data. This emphasizes the importance of proper password handling and secure authentication practices in real-world applications.

VII. Conclusion

The PW Crack 1 challenge reinforced fundamental skills in **source code analysis, file handling, and basic password security concepts**. By understanding how authentication logic is implemented in a script, the correct password was identified without the need for advanced cracking techniques. This challenge serves as a reminder of the importance of strong password management and secure coding practices in cybersecurity.

— **The Analyst: Hyposelenia**